Project preview & user manual

Rea-Sixty

An open-source REAPER extension for the SSL UF8 and UC1.

Prepared for Solid State Logic as a courtesy preview ahead of any formal correspondence. The aim is transparency: what the project is, how it works, what it does today, and what it asks of SSL — nothing more.

Project	Rea-Sixty (repository: *reaper-uf8*)
Type	REAPER extension — C++, csurf_inst, libusb
Status	Working, near feature-complete on macOS (Phase 1 + Phase 2)
Source	github.com/acklin83/reaper-uf8
License	MIT — original code only, no SSL material redistributed
Document date	2026-05-09

Contents

1. Executive summary

Rea-Sixty is an independent open-source REAPER extension that drives the SSL UF8 and UC1 directly from REAPER. It opens the controllers' vendor-USB interface, replays the host-side init sequence, and emits the same Plug-in-Mixer-mode frames that SSL 360° produces — reading track state from REAPER's C API (name, colour, fader, pan, sends, automation, focused FX, gain reduction). The result: REAPER users get the colour-aware scribble-strip experience natively, without an SSL VST3 on every track, and the SSL Bus Compressor's GR meter on the UC1 follows audio-driven gain reduction on the focused track.

The extension is a single shared library (`reaper_uf8.dylib` on macOS), MIT-licensed. It does not redistribute, decompile, or reproduce any SSL software, plug-in, firmware, or trademark; all interoperability is established by passive USB observation of legally purchased SSL hardware running legally licensed SSL 360°.

2. Why this exists

The SSL UF8's scribble strips can render DAW track colours, but only when the controller is in Plug-in Mixer Mode — which in turn requires an SSL VST3 (Channel Strip 2 / 4K B / E / G, 360 Link, or Bus Compressor) on every track that should appear. For 100+ track REAPER sessions this is impractical, and it is the most frequent feature request we hear from REAPER users with UF8s.

MCU and HUI cannot carry colour at all, so any “just talk MCU” workaround is dead on arrival for the colour use case. And SSL 360° holds the UF8's vendor-USB interface with an exclusive claim, so a quiet side-car next to 360° is not possible either. The only technically honest path was to re-implement 360°'s host-side responsibilities for REAPER specifically — which is exactly what Rea-Sixty does.

3. How it works

Rea-Sixty registers with REAPER as a `csurf_inst` (`IReaperControlSurface`). On load it enumerates SSL devices via libusb (VID `0x31E9`, PIDs `0x0021` UF8 / `0x0023` UC1), claims the vendor interface, and replays the init sequence we observed 360° sending at startup. It then runs the keepalive cycle the controllers require to stay in Plug-in Mixer mode, and pushes frames driven by REAPER state on a single timer.

Inputs from the controller (button events, fader motion + touch, V-Pot deltas, UC1 knob deltas) are parsed from vendor-USB packets and dispatched through the REAPER C API (`CSurf_OnVolumeChange`, `CSurf_OnMuteChange`, `Main_OnCommand`, `TrackFX_SetParam`, …). No virtual MIDI port, no Control Surface Integrator, no MCU bridge.

Topology

```
REAPER <-> Rea-Sixty extension (csurf_inst) <-> UF8 / UC1 (vendor-USB, libusb)
```

Specifically *not*:

```
REAPER <-> CSI <-> virtual MCU MIDI <-> bridge <-> UF8
```

Decoded protocol surface

The following are implemented and verified on the hardware:

- Frame format and checksum on both endpoints (`FF <cmd> <len> <data> <chk>`).

- UF8 vendor init sequence + Plug-in-Mixer keepalive pair (13 B / 64 B cycling, ~150 ms).
- Per-strip TFT colour command and the 16-entry palette (12 of 16 indices mapped; remainder snap nearest-RGB).
- Per-strip SEL / MUTE / SOLO LED frames; transport LEDs; 12-segment metering frame.
- Display zones: scribble text, parameter label, value line, channel-strip type, O/PdB readout, channel-number, color bar.
- UF8 inbound button-ID map (per-strip + globals, full table on page 9).
- UC1 inbound button + knob ID map; outbound LED frames per cell; GR meter (16-bit BE dB×10), VU bands, display zones.

4. What works today

Status as of the document date. macOS is the lead platform; Windows and Linux ports are tracked but not yet shipped.

Capability	State	Notes
UF8 standalone replacement	Shipping	Phase 1 complete — no SSL 360° required for UF8.
DAW-layer scribble strip colours	Shipping	Polled from `GetTrackColor()`; pushed on bank shifts. Not offered by SSL 360°.
Full 16-bit fader resolution	Shipping	Direct `CSurf_OnVolumeChange`; no MCU 14-bit lossy stage.
UC1 parameter mirror (focused FX)	Shipping	Knobs follow Channel Strip 2 / Bus Comp 2 on the focused track; values mirrored on the UC1 displays.
UC1 GR display (audio-driven)	Shipping	Driven by `TrackFX_GetNamedConfigParm("GainReduction_dB", …)` — the PreSonus VST3 GR extension that REAPER exposes for the SSL plug-ins.
Bindings (per-strip / transport / global / soft-keys / Learn)	Shipping	12 user soft-key banks + the 6 SSL stock banks; modifier combos.
Generic FX-parameter learn	Shipping	GUID-keyed; survives FX-slot reorders. Works for any VST/JS/AU param.
On-screen Plug-in Mixer + Settings	Shipping	Vendored Dear ImGui in a dockable SWELL window; picks up the user's REAPER theme.
Folder mode + selection sets + send/receive layers	Landing	Phase 2.5 in progress.
Windows / Linux ports	Planned	Phase 4. Capture workflow already runs on Windows.

Known limitations

- SSL 360° must be quit before REAPER starts — the exclusive USB claim is unchanged.
- Some palette indices (likely greys) still snap to nearest match instead of being decoded.
- Coexistence with 360° is not pursued; a complete replacement is the explicit goal.

- Hardware behaviour under unforeseen frames is not part of SSL's documented public API; we ship with a clear “use at your own risk — may void warranty” notice (see Legal).

5. Legal posture

We took care to be on solid legal footing before publishing.

Trademarks

“SSL”, “Solid State Logic”, “SSL 360°”, “UF8”, and “UC1” are trademarks of Solid State Logic Ltd. They are used in the project documentation solely to identify the hardware and software this extension interoperates with (nominative fair use). The project is not affiliated with, endorsed by, or sponsored by SSL.

Interoperability basis

Developed via independent passive observation of the USB wire protocol between legally purchased SSL UF8 / UC1 hardware and legally licensed SSL 360° software, for the sole purpose of achieving interoperability with REAPER. No SSL code, firmware, binaries, or proprietary creative content is decompiled, reproduced, or redistributed. Cited authorities: EU Software Directive 2009/24/EC Art. 6 (interoperability exception); §69e UrhG (Germany); 17 USC §1201(f) (US interoperability exception).

No warranty

The project ships under MIT with an explicit “use at your own risk” notice that running third-party firmware-level communication with SSL hardware may void the user's warranty.

6. Asks of SSL

We would much rather collaborate than work in parallel. Concretely:

1. — Any objection to this being a public, open-source project? We would rather hear it now than after the fact, and we are open to changing how the project is shaped.

2. — Would SSL be willing to share protocol documentation, under NDA if needed? Even partial reference would save us substantial capture-and-decode effort and produce a more robust result for shared customers.

3. — If this could grow into something more formal — a documented partner extension, a listing alongside SSL's supported integrations — we'd be glad to talk.

We did this because we love the hardware and were curious what was possible. Collaboration is the preferred outcome.

USER MANUAL (preview)

Rea-Sixty for REAPER

The following sections are the user-facing manual that ships with the extension. Included here as a preview so SSL can see exactly what end-users are told to do.

7. Installation

Prerequisites

- REAPER (any recent version, 6.x / 7.x).
- An SSL UF8, UC1, or both, connected via USB.
- macOS: Homebrew — `brew install libusb cmake pkg-config`.
- **SSL 360° must be quit before REAPER starts.** 360° holds the controller's vendor interface exclusively; Rea-Sixty cannot open it while 360° is running.

Build (macOS)

```
git clone https://github.com/acklin83/reaper-uf8.git
cd reaper-uf8/extension
cmake -B build -G "Unix Makefiles"
cmake --build build -j$(sysctl -n hw.ncpu)
```

Install into REAPER

Symlink (preferred during development — rebuilds are live):

```
ln -sf "$PWD/build/reaper_uf8.dylib" \
~/Library/Application\ Support/REAPER/UserPlugins/reaper_uf8.dylib
```

Or copy:

```
cp build/reaper_uf8.dylib \
~/Library/Application\ Support/REAPER/UserPlugins/
```

Restart REAPER.

8. First run — what to expect

- On load, nothing is visible inside REAPER itself. The extension is a control-surface plug-in; there is no menu entry to open.
- The UF8 wakes from the “Awaiting Connection to SSL 360° Software” idle screen and enters Plug-in Mixer Mode.
- Scribble strips show the first eight tracks: track number, name, fader dB readout, V-Pot ring (Pan by default), and the DAW track colour bar.
- The 12-segment meter on each strip follows REAPER's track meter.
- If a UC1 is connected, its knobs follow the focused track's SSL Channel Strip 2 / Bus Compressor 2 (whichever is present); the GR meter follows the plug-in's gain reduction.
- If something fails, REAPER's Console (View → Console) shows `reaper_uf8: <reason>`. The most common reason is `SSL360Core owns the device`.

9. Day-to-day operation

The control surface behaves the way a UF8/UC1 owner already expects, with a few REAPER-native specifics:

Control	Default behaviour
Fader	REAPER track volume, full 16-bit. Touch is detected; automation latches honour REAPER's automation modes.
V-Pot rotate	Pan by default. *Flip* swaps with the fader. *Plugin* targets the focused FX param.
V-Pot push	Reset target to default (pan = 0; param = default value).
SOLO / CUT / SEL	Solo / Mute / Select on the strip's track. SEL behaves as “exclusive select” on a single press (no MCU double-press needed).
BANK < / >	Shift visible 8 by 8 tracks. CHANNEL < / > shifts by 1.
LAYER 1 / 2 / 3	Cycle the user-defined layer (DAW / FX / Sends, configurable).
QUICK 1 / 2 / 3	Plug-in Mixer assignments: 1 = Channel Strip, 2 = Bus Comp, 3 = I/O meter toggle (matches the SSL stock layout).
Soft keys (above strips)	Per-strip parameter-page key in PM mode; bind to any REAPER action via Learn Mode.
Transport	Standard REAPER Play / Stop / Record / Rewind / Fast-forward.
CHANNEL encoder	Standard = bank ±1; NAV = scrub; NUDGE; FOCUS = mouse-wheel emulation. Press-and-hold = Cursor-Transport mode.
360° key	Opens the on-screen Plug-in Mixer + Settings window (Rea-Sixty's, not SSL 360°'s).
UC1 knobs	Drive the focused track's Channel Strip / Bus Compressor in real time. *Fine* halves the step size.
UC1 GR meter	Audio-driven gain reduction for the focused track's SSL compressor.

10. UF8 button & LED reference

The IDs below are the values the UF8 emits on its vendor-USB IN endpoint — documented here so users writing their own ReaScript automations can identify which physical control fired. Per-strip indices are `N = 0..7`, left to right.

Per-strip buttons

Button	Formula	Strip 0	Strip 7
V-Pot push	0x08 + N	0x08	0x0F
Top soft-key (above scribble)	0x18 + N	0x18	0x1F
SOLO	0x20 + 3·N	0x20	0x35
CUT (Mute)	0x21 + 3·N	0x21	0x36

Button	Formula	Strip 0	Strip 7
SEL (Select)	0x22 + 3·N	0x22	0x37

Global buttons (selection)

ID	Name	ID	Name
0x40	Layer 1	0x41	Layer 2
0x42	Layer 3	0x43	Quick 1 (Channel Strip)
0x44	Quick 2 (Bus Comp)	0x45	Quick 3 (I/O meter)
0x46	360° / Settings	0x50	Plugin
0x51	Channel	0x52	Page ←
0x53	Page →	0x54	Flip
0x58	Automation Off	0x59	Read
0x5A	Write	0x5B	Trim
0x5C	Latch	0x5D	Touch
0x6E	PAN	0x6F	Fine / Shift
0x70	Norm / Clear	0x71	Rec / All
0x72	Auto / Zero	0x76	Channel encoder push
0x78	Bank ←	0x79	Bank →

Outbound LED / colour

Function	Frame
Per-strip TFT colour	`FF 66 09 18 <8 palette indices> <chk>`
Per-strip SEL / MUTE / SOLO LED	`FF 3B 03 <id> 00 <state> <chk>`
12-segment track meter	`FF 38 04 … / FF 39 04 …`
Layer / page (also PM keepalive)	`FF 1B 01 <XX> <chk>`

11. UC1 reference

UC1 button IDs are stable across plug-in contexts (Channel Strip vs Bus Comp). Knob IDs in the dedicated EQ / Dynamics zone always map to their named function. The top-centre V-Pot row is soft-mapped per focused plug-in.

Buttons

ID	Button	ID	Button
0x08	HF Bell	0x09	EQ Type
0x0A	EQ In	0x0B	LF Bell
0x0C	Bus Comp IN	0x14	Fast Attack (Comp)
0x15	Peak	0x16	Dyn In
0x17	Expand	0x18	Fast Attack (Gate)
0x19	Polarity	0x1A	S/C Listen
0x1B	Solo Clear	0x1C	Solo
0x1D	Cut	0x1E	Channel IN
0x1F	Fine		

Channel-strip knobs (always CS params)

ID	Knob	ID	Knob
0x00	LPF freq	0x07	LMF Gain
0x01	HPF freq	0x08	LMF Frequency
0x02	HF Gain	0x09	LMF Q
0x03	HF Frequency	0x0A	LF Frequency
0x04	HMF Gain	0x0B	LF Gain
0x05	HMF Frequency	0x17	Gate Release
0x06	HMF Q	0x18	Gate Hold
0x19	Gate Threshold	0x1A	Gate Range
0x1B	Comp Release	0x1C	Comp Threshold
0x1D	Comp Ratio		

Outbound (selected)

Function	Frame
GR meter	`FF 5B 02 <BE-16 dB×10> <chk>`
VU meter	`FF 13 04 01 <level> 01 <in/out>`
Per-button LED	`FF 13 04 <bank> <cell> 01 <state>`
Display zone text	`FF 66 <len> <zone> <ascii…> <chk>`
Keepalive (required)	`FF 1B 01 <counter> <chk>` (~1 Hz)

12. Bindings & Learn Mode

Mappings live in a single JSON file. The on-screen Settings screen and the Learn Mode workflow both edit it; the extension watches it on disk and reloads on change — no REAPER restart needed.

Config file location

OS	Path
macOS	`~/Library/Application Support/REAPER/rea_sixty/bindings.json`
Windows	`%APPDATA%\REAPER\rea_sixty\bindings.json`
Linux	`~/.config/REAPER/rea_sixty/bindings.json`

Binding types

- **reaper_action** — dispatches a REAPER action (numeric ID or named SWS / ReaPack command).
- **builtin** — surface behaviour (bank navigation, layer cycle, V-Pot mode, flip, mute/solo clear, …).
- **track_target** — per-strip standard target (select, mute, solo, rec_arm, phase, monitor, fx_bypass, automation_mode).
- **fx_param** — bind a V-Pot or soft-key to an FX parameter, GUID-keyed so FX-slot reorders don't break the mapping.

Learn Mode

- Open the Settings screen, click **Learn**.
- Press the UF8 / UC1 control you want to bind. The extension reports the captured control back to the UI.
- Trigger the REAPER action (or move the FX parameter) you want to bind to it.
- The binding is written to `bindings.json` and the extension reloads it.
- ESC at any time cancels Learn Mode.

13. Plug-in Mixer window & Settings

The on-screen Plug-in Mixer mirrors the eight currently-banked tracks — track name, colour, fader, pan, focused FX parameter — in a dockable window inside REAPER. It uses a vendored Dear ImGui inside a SWELL host window, and picks up the user's REAPER theme automatically (light or dark).

The Settings screen lives in the same window. It exposes controller brightness, sleep timeout, default V-Pot mode per layer, the soft-key bank editor, and the Learn-Mode workflow. All settings persist to the same JSON file the extension reads at startup.

14. Troubleshooting

Symptom	Cause / fix
UF8 stays on the “Awaiting Connection to SSL 360°” screen.	SSL 360° is still running. Quit it, then restart REAPER.

Symptom	Cause / fix
REAPER Console: `SSL360Core owns the device`.	Same as above — the vendor interface is exclusively claimed.
Scribble strips light up but stay blank.	Init replayed but Plug-in Mixer keepalive lost. Check the Console for libusb timeout errors and reseat the USB cable.
Track colour on the strip is “close but not exact”.	REAPER colour is being snapped to the nearest 360° palette entry. Some palette indices are still unmapped; the snap is by design.
UC1 GR meter sits at zero with audio playing.	The focused track has no SSL Bus Comp / Channel Strip, or the plug-in's GR readout extension is unavailable in this REAPER build.
Fader feels less responsive than under SSL 360°.	Should not happen — we use full 16-bit. If it does, please open a GitHub issue with a session description.
UF8 LCDs show no colour bar despite seeing the strips.	The colour command sits in PM mode only. Confirm the layer is not set to Plug-in Mixer Off.

Anything not on the list goes to the issue tracker:

`github.com/acklin83/reaper-uf8/issues`

15. Uninstall

Remove the symlink (or copy) from REAPER's UserPlugins folder and restart REAPER:

```
rm ~/Library/Application\ Support/REAPER/UserPlugins/reaper_uf8.dylib
```

The bindings JSON is left in place so re-installing later preserves the user's mappings. Delete the `rea_sixty` folder under REAPER's resource directory to remove it as well.

End of preview

This document is a snapshot for SSL of the project as it stands today. The living source of truth is the GitHub repository: `github.com/acklin83/reaper-uf8`.

Questions, objections, or an opening to collaborate — all very welcome. See section 6 for the asks; the outreach email draft is in `docs/outreach/` in the repository.